

POKÉ VAULT

COMMERCE

ระบบจัดการร้านค้า Pokémon TCG Pocket

CP353002 • SPRING BOOT



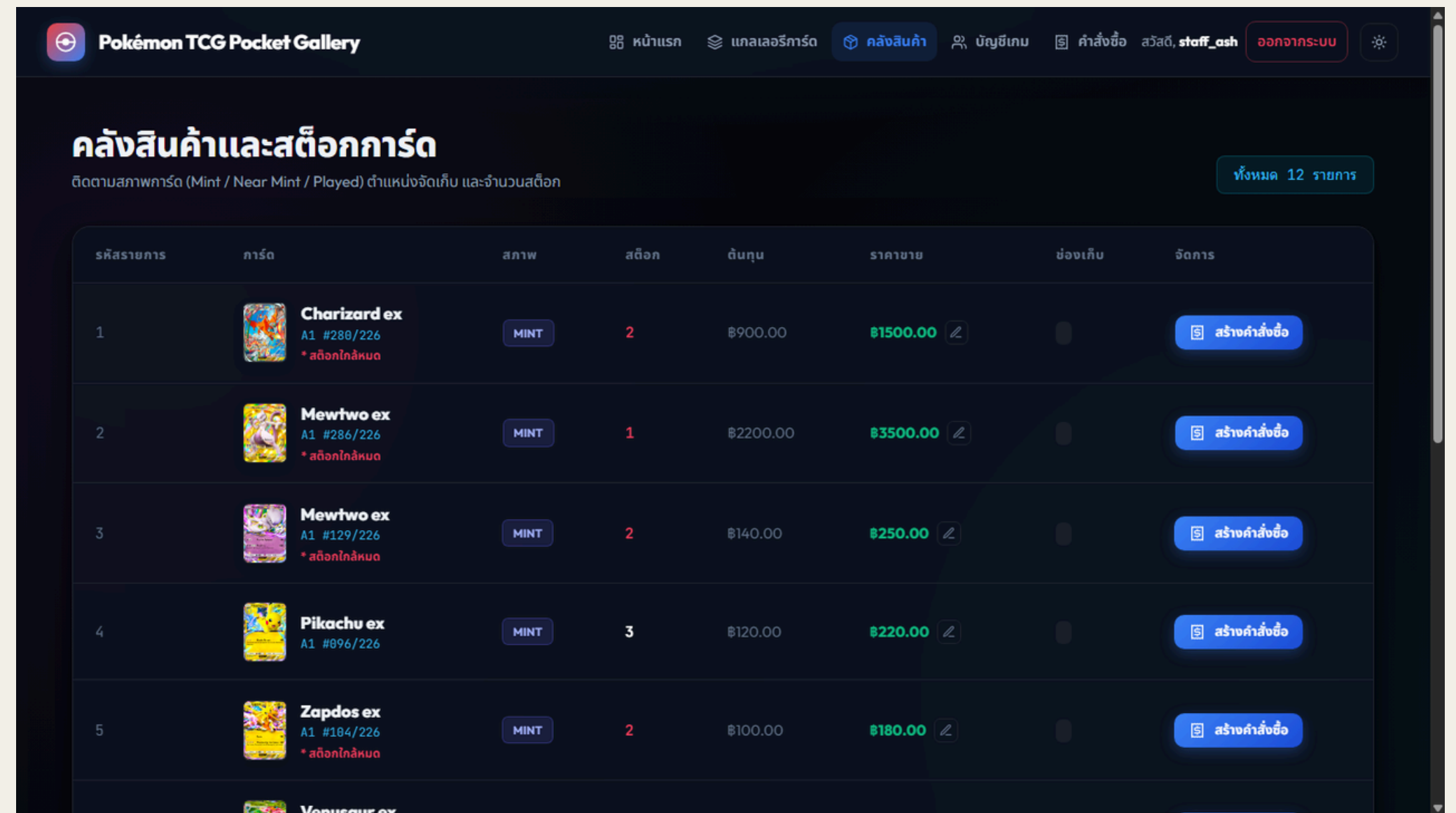
ปัญหาของร้านค้าการ์ด

01 การ์ดกระจายหลายบัญชี

02 ตรวจสอบสต็อกและเลือกบัญชีส่ง ยาก

03 ติดตามการส่งไม่ครบ

แคตตาล็อก → สต็อก → คำสั่งซื้อ → เทรด
การส่งการ์ดยังทำในเกม

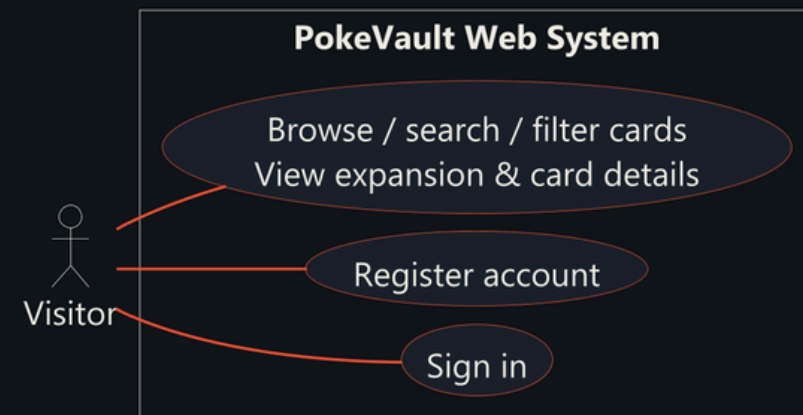


The screenshot shows the 'Pokémon TCG Pocket Gallery' website. The main heading is 'คลังสินค้าและสต็อกการ์ด' (Inventory and Card Stock). Below it, a subtitle reads 'ติดตามสภาพการ์ด (Mint / Near Mint / Played) ตำแหน่งจัดเก็บ และจำนวนสต็อก' (Track card condition, storage position, and stock quantity). A button in the top right corner says 'ทั้งหมด 12 รายการ' (Total 12 items). The table below lists five cards for sale:

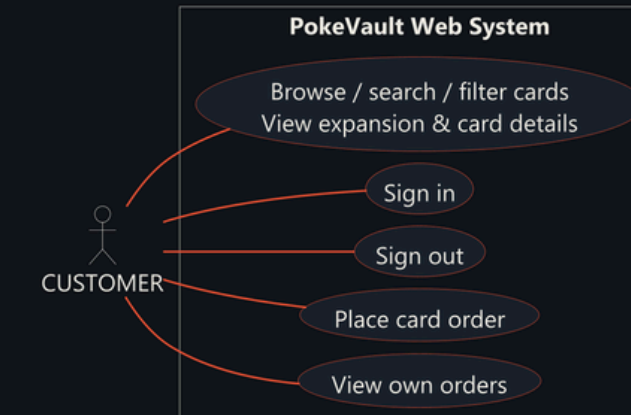
รหัสรายการ	การ์ด	สภาพ	สต็อก	ต้นทุน	ราคาขาย	ช่องเก็บ	จัดการ
1	 Charizard ex A1 #288/226 * สต็อกใกล้หมด	MINT	2	฿900.00	฿1500.00		 สร้างคำสั่งซื้อ
2	 Mewtwo ex A1 #286/226 * สต็อกใกล้หมด	MINT	1	฿2200.00	฿3500.00		 สร้างคำสั่งซื้อ
3	 Mewtwo ex A1 #129/226 * สต็อกใกล้หมด	MINT	2	฿140.00	฿250.00		 สร้างคำสั่งซื้อ
4	 Pikachu ex A1 #896/226	MINT	3	฿120.00	฿220.00		 สร้างคำสั่งซื้อ
5	 Zapdos ex A1 #184/226 * สต็อกใกล้หมด	MINT	2	฿100.00	฿180.00		 สร้างคำสั่งซื้อ
	 Venusaur ex						

អ្នកប្រើប្រាស់ និង Use Case

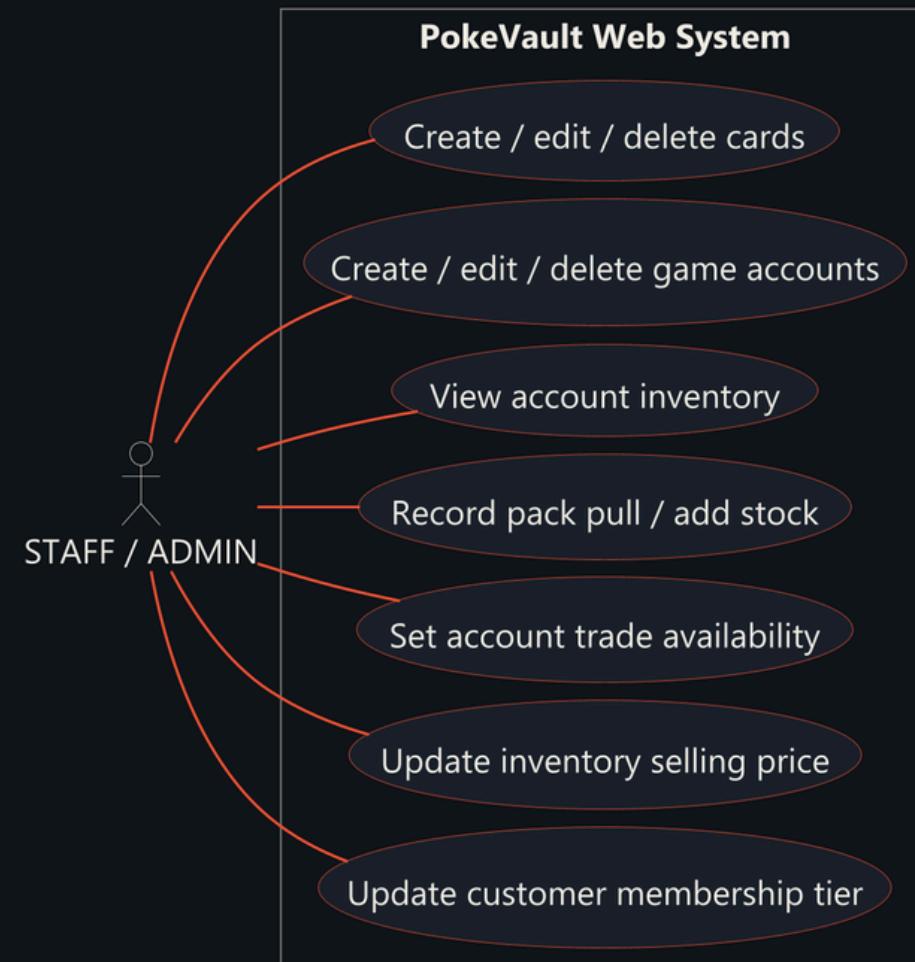
01 PUBLIC ACCESS



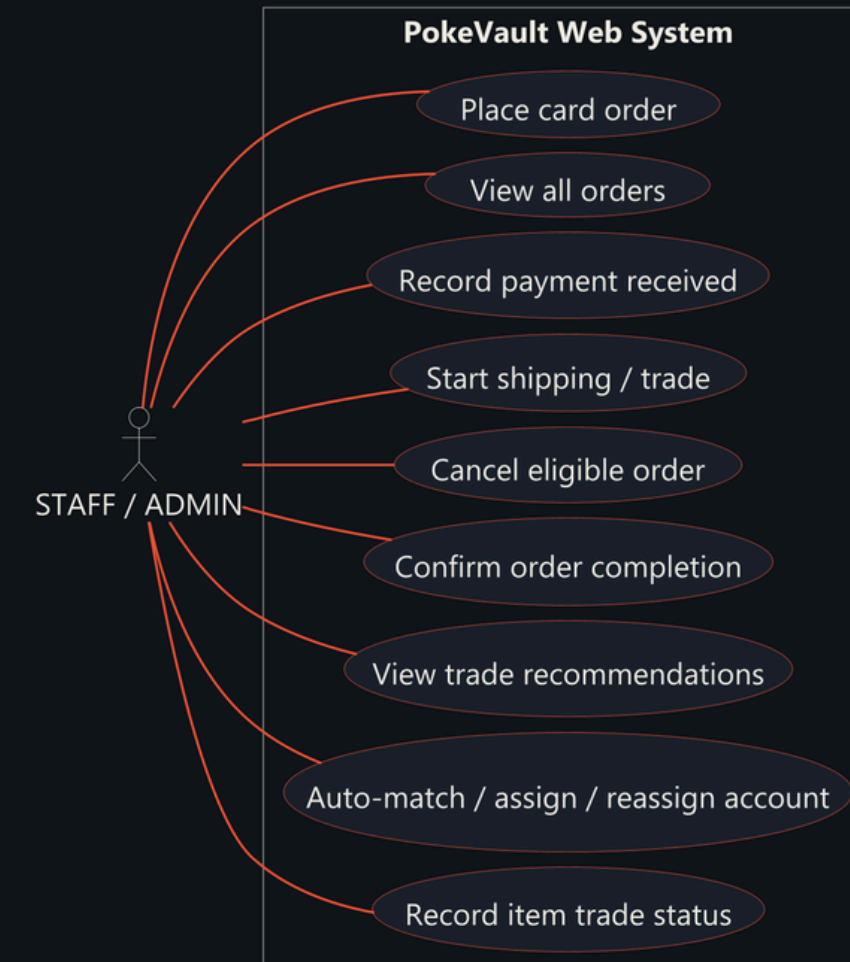
02 CUSTOMER



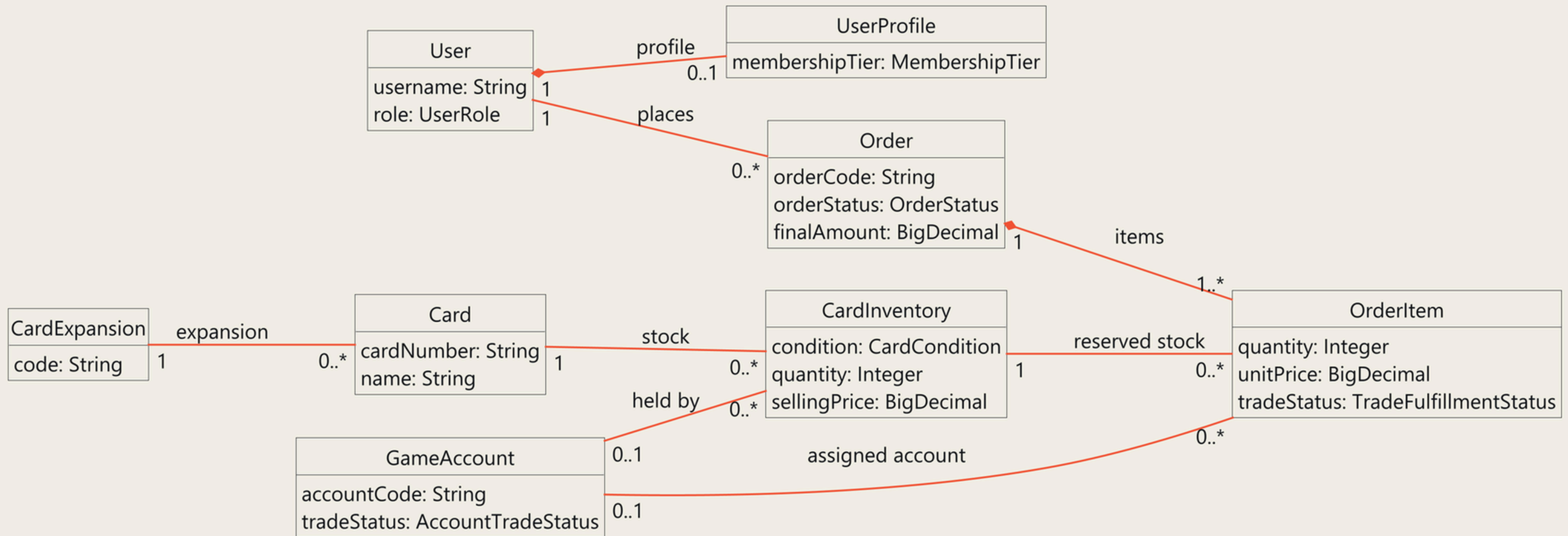
03 CATALOG & VAULT



04 ORDER & TRADE

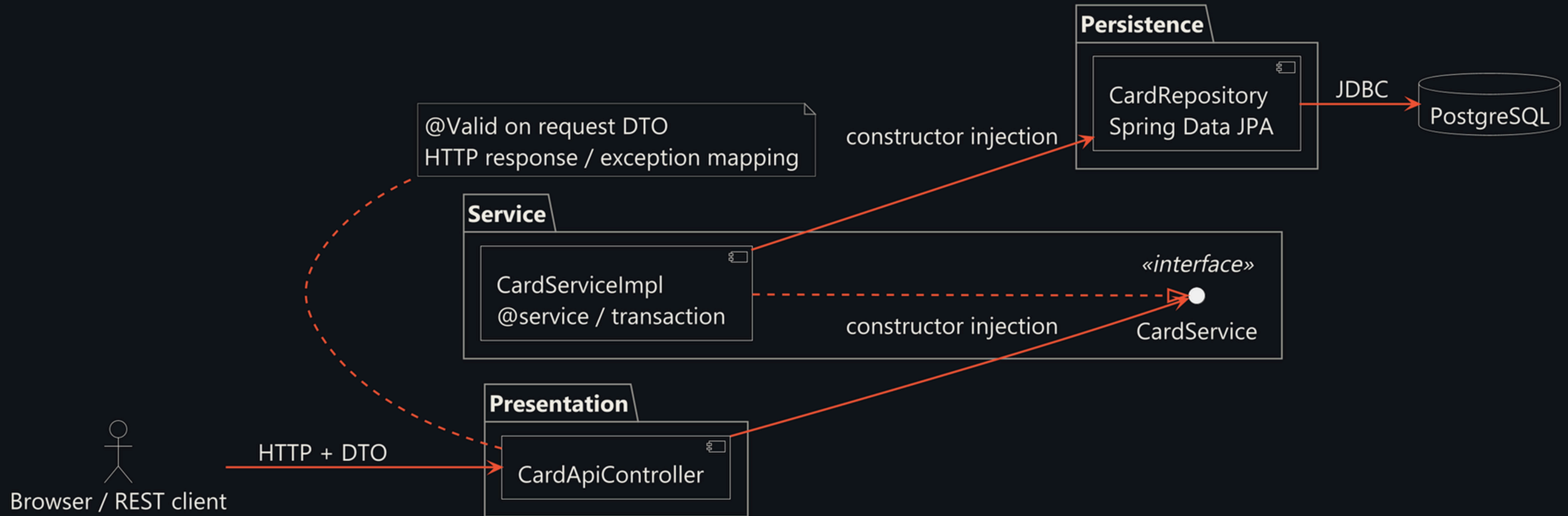


ภาพย่อ: Domain และตำแหน่ง Patterns



Strategy: DiscountStrategy • State: OrderState • Observer: LowStockObserver

Layered Architecture



Controller เรียก Service interface • constructor injection • Repository รับผิดชอบ persistence

Layered: จาก Request ถึง Repository

CardApiController.javaL85-89

85 public ResponseEntity<ApiResponse<CardResponse>> createCard(@Valid @RequestBody
CardRequest request) {
86 CardResponse created = cardService.createCard(request);
87 return ResponseEntity.status(HttpStatus.CREATED)
88 .body(ApiResponse.ok("Card created successfully", created));
89 }

CardServiceImpl.javaL171-175, 188-190

171 @Transactional
172 public CardResponse createCard(CardRequest request) {
173 CardExpansion expansion =
cardExpansionRepository.findById(request.getExpansionId())
174 .orElseThrow(() -> new ResourceNotFoundException("CardExpansion",
"id", request.getExpansionId()));
175
บรรทัด 176-187 ไม่แสดงในภาพนี้
188 Card savedCard = cardRepository.save(card);
189 return CardResponse.fromEntity(savedCard);
190 }

01 Controller รับและตอบ HTTP

รับ CardRequest ผ่าน @Valid
เรียก cardService และตอบ 201

02 Service ตรวจสอบข้อมูลอ้างอิง

ตรวจสอบว่า Expansion มีอยู่จริง
และควบคุม Transaction

03 Repository บันทึกข้อมูล

Service เรียก save()
แล้วแปลง Entity เป็น CardResponse

ตรงกับแผนภาพ Request: Controller เรียก Service และ Service เรียก Repository

SOLID ในโค้ด

S

แยกหน้าที่
Controller / Service
/ Repository

O

เพิ่มสูตรส่วนลด
DiscountStrategy

L

Regular / Vip /
Wholesale
ใช้ contract เดียวกัน

I

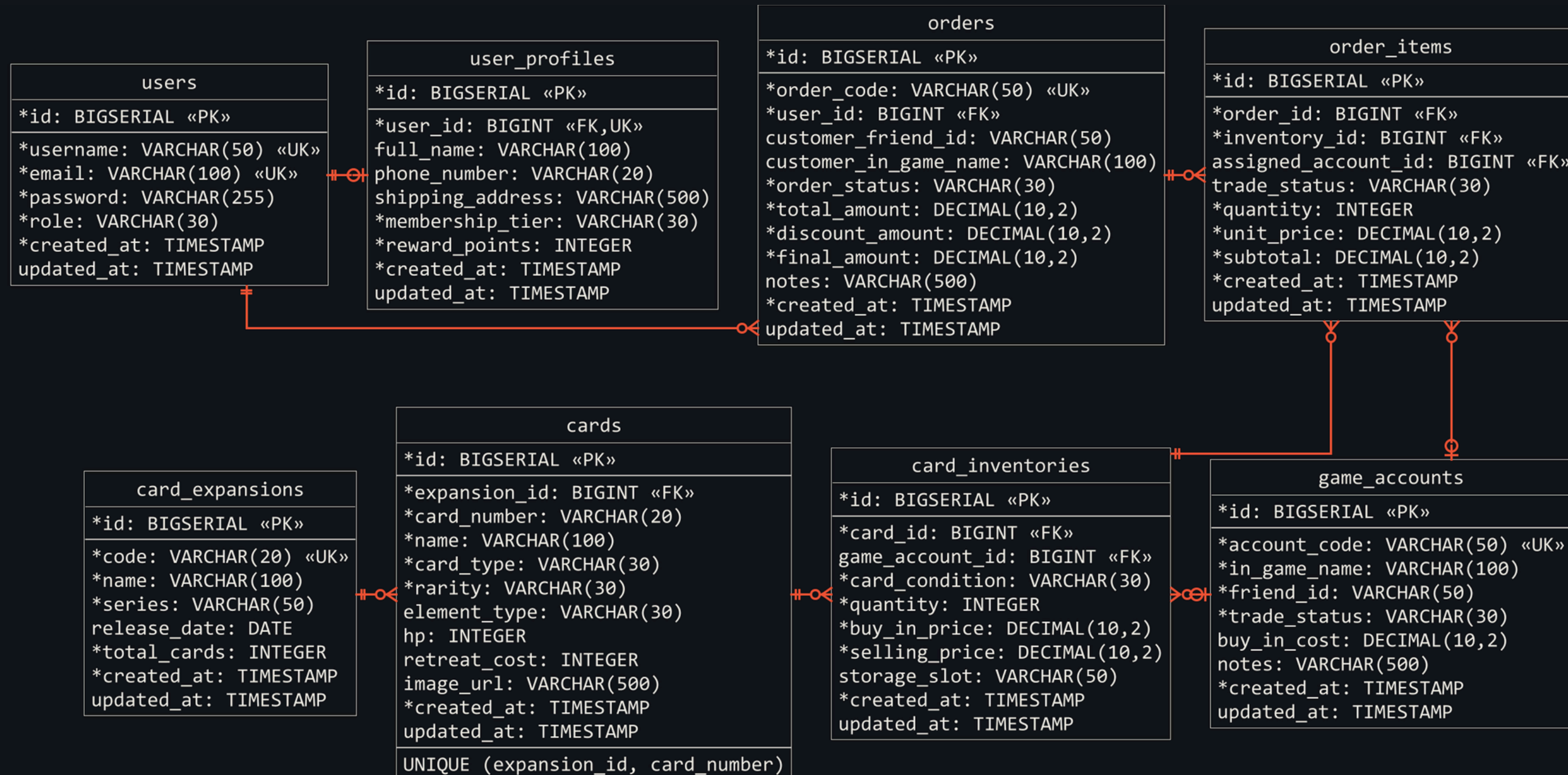
CardService แยก
จากงานบัญชีเกม

D

Controller พึ่ง
Service interface
ผ่าน constructor

หมายเหตุ: SecurityConfig ยังพึ่ง CustomUserDetailsService โดยตรง • source: doc/solid-analysis.md

Database Design



• NOT NULL | PK Primary key | FK Foreign key | UK Unique key
Crow's foot: || exactly one | o| zero or one | o{ zero or many
SQL permits 0 order items; the booking API requires at least 1 item.

Patterns 01–03

LAYERED

แยก HTTP / ธุรกิจ / Persistence

Controller → Service → Repository
เปลี่ยน UI หรือ DB ได้โดยไม่รวม
กฎธุรกิจไว้ใน Controller

MVC

แยกข้อมูลและการแสดงผล

WebViewController + Spring Model
+ Thymeleaf templates
View ไม่รับผิดชอบ transaction

REPOSITORY

รวมสัญญาการเข้าถึงข้อมูล

CardRepository / OrderRepository
extends JpaRepository
Service ใช้ query/save ผ่าน interface

Patterns 04–06

SERVICE LAYER

ควบคุม Use Case ในจุดเดียว

OrderServiceImpl @Transactional
ตรวจสอบผู้ใช้ → สต็อก → ส่วนลด → save
เกิดข้อผิดพลาดแล้ว rollback

DTO + MAPPER

แยกสัญญาณภายนอกจาก Entity

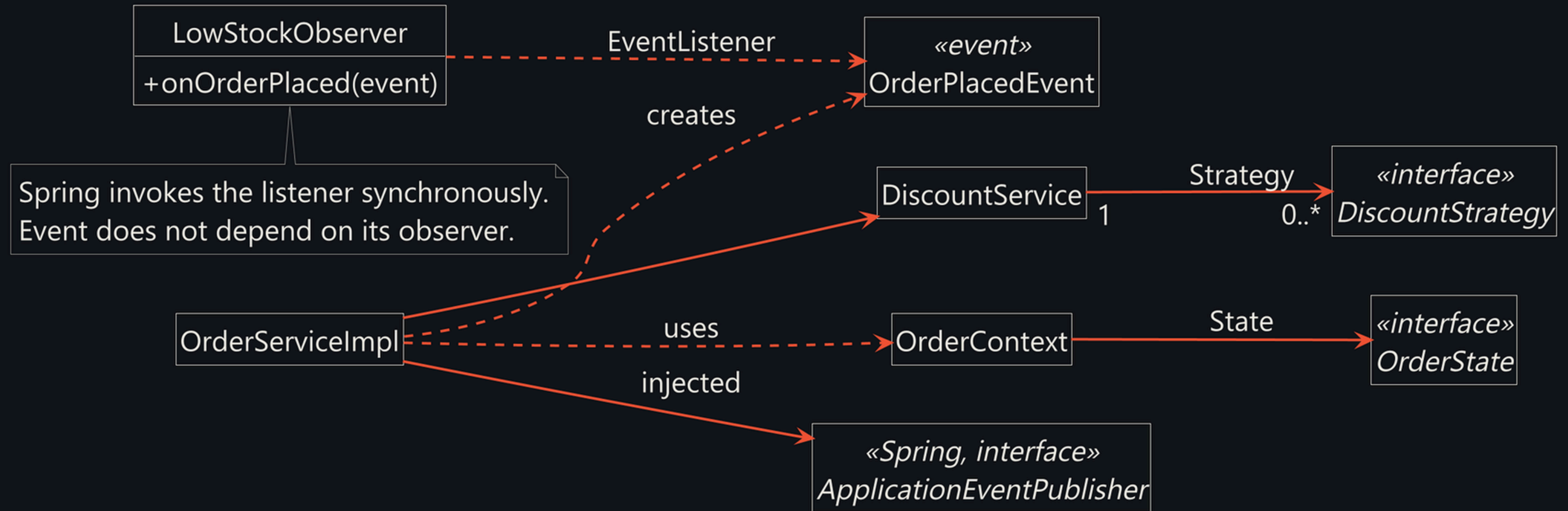
CardRequest / CardResponse
OrderResponse / WebViewModelapper
คุมข้อมูลที่รับและส่ง

DI

แสดง Dependency ชัดเจน

final fields + Constructor
Spring จัดหา Service / Repository
เปลี่ยน dependency เป็น mock ได้

Class Diagram: ตำแหน่ง Patterns



EventListener ฟังหา OrderPlacedEvent • Event ไม่ได้ฟังหา Observer

Strategy: เลือกสูตรส่วนลดสมาชิก

DiscountService.java

L14-17, 28-31

```

14 private final List<DiscountStrategy> strategies;
15
16 public DiscountService(List<DiscountStrategy> strategies) {
17     this.strategies = strategies != null ? strategies : List.of();
    บรรทัด 18-27 ไม่แสดงในภาพนี้
28 public DiscountStrategy getApplicableStrategy(MembershipTier tier) {
29     return strategies.stream()
30         .filter(strategy -> strategy.supports(tier))
31         .findFirst()

```

VipDiscountStrategy.java

L12, 16-20

```

12 private static final BigDecimal DISCOUNT_RATE = new BigDecimal("0.10");
    บรรทัด 13-15 ไม่แสดงในภาพนี้
16 public BigDecimal calculate(BigDecimal subtotal) {
17     if (subtotal == null || subtotal.compareTo(BigDecimal.ZERO) <= 0) {
18         return BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP);
19     }
20     return subtotal.multiply(DISCOUNT_RATE).setScale(2, RoundingMode.HALF_UP);

```

สูตรส่วนลดแยกอยู่ในแต่ละ **Strategy** ส่วน **Service** ทำหน้าที่เลือกและเรียกใช้งาน

01 รับ Strategy ด้วย DI

Spring ส่ง List<DiscountStrategy>
ให้ DiscountService

02 เลือกตาม Membership Tier

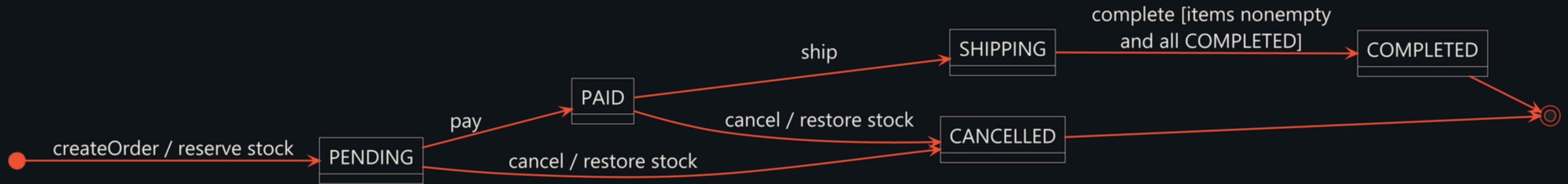
supports(tier) ระบุว่า
Strategy ไดรองรับสมาชิกรันั้น

03 สูตร VIP ลด 10%

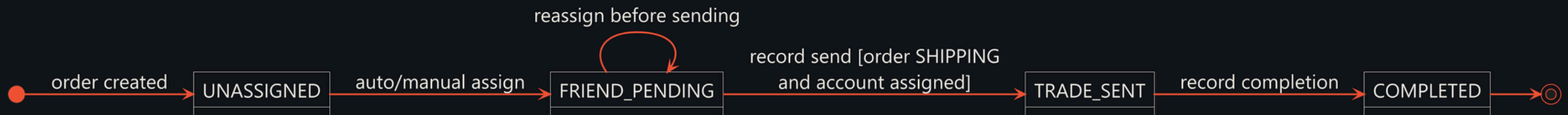
คำนวณด้วย BigDecimal
และปัดทศนิยมสองตำแหน่ง

Order และ Item States

ORDER



ORDER ITEM



รายการสุดท้าย **COMPLETED** / ปิดออเดอร์อัตโนมัติ • **SHIPPING** ยกเลิกไม่ได้

State: เงื่อนไขการปิดคำสั่งซื้อ

ShippingOrderState.java

L12-26

```

12 public void complete(OrderContext context) {
13     if (context != null && context.getOrder() != null) {
14         Order order = context.getOrder();
15         if (order.getItems() == null || order.getItems().isEmpty()) {
16             throw new TradeStateConflictException("Cannot complete order: order
has no items");
17         }
18         boolean allItemsCompleted = order.getItems().stream()
19             .allMatch(item -> item.getTradeStatus() ==
TradeFulfillmentStatus.COMPLETED);
20         if (!allItemsCompleted) {
21             throw new TradeStateConflictException(
22                 "Cannot complete order: not all items have reached
COMPLETED trade status");
23         }
24     }
25     // เมื่อการเทรดในเกมเสร็จสิ้นสมบูรณ์ -> สลับไปสถานะ COMPLETED
26     context.setState(new CompletedOrderState());

```

01 ตรวจสอบสถานะทุกรายการ

allMatch() ตรวจสอบว่า OrderItem
ทุกใบเป็น COMPLETED

02 ยังเทรดไม่ครบ

โยน TradeStateConflictException
เมื่อมีรายการที่ยังไม่เสร็จ

03 เปลี่ยนพฤติกรรมตาม State

เมื่อผ่านเงื่อนไขแล้วจึงเปลี่ยน
เป็น CompletedOrderState

Order และ OrderItem มีสถานะแยกกัน การปิดออเดอร์ต้องผ่านกฎใน ShippingOrderState

Observer: ตรวจสอบต่อจาก Event

OrderServiceImpl.javaL141-146

141eventPublisher.publishEvent(OrderPlacedEvent.builder()
142.orderId(savedOrder.getId())
143.orderCode(savedOrder.getOrderCode())
144.items(savedOrder.getItems())
145.timestamp(LocalDateTime.now())
146.build());

LowStockObserver.javaL38-39, 50-55

38@EventListener
39public void onOrderPlaced(OrderPlacedEvent event) {
 บรรทัด 40-49 ไม่แสดงในภาพนี้
50Card card = inventory.getCard();
51int remainingStock =
cardInventoryRepository.sumQuantityByCardId(card.getId());
52
53if (remainingStock <= LOW_STOCK_THRESHOLD) {
54log.warn("⚠️ LOW STOCK ALERT: Card '{}' ({{}}) has only {{
copies left in vault (threshold: {{}})!",
55card.getName(), card.getCardNumber(), remainingStock,
LOW_STOCK_THRESHOLD);

Order ส่งเหตุการณ์โดยไม่เรียก LowStockObserver โดยตรง การแจ้งเตือนปัจจุบันอยู่ใน Log

01 Order ส่ง Event

publishEvent() ส่ง orderId
orderCode และรายการการ์ด

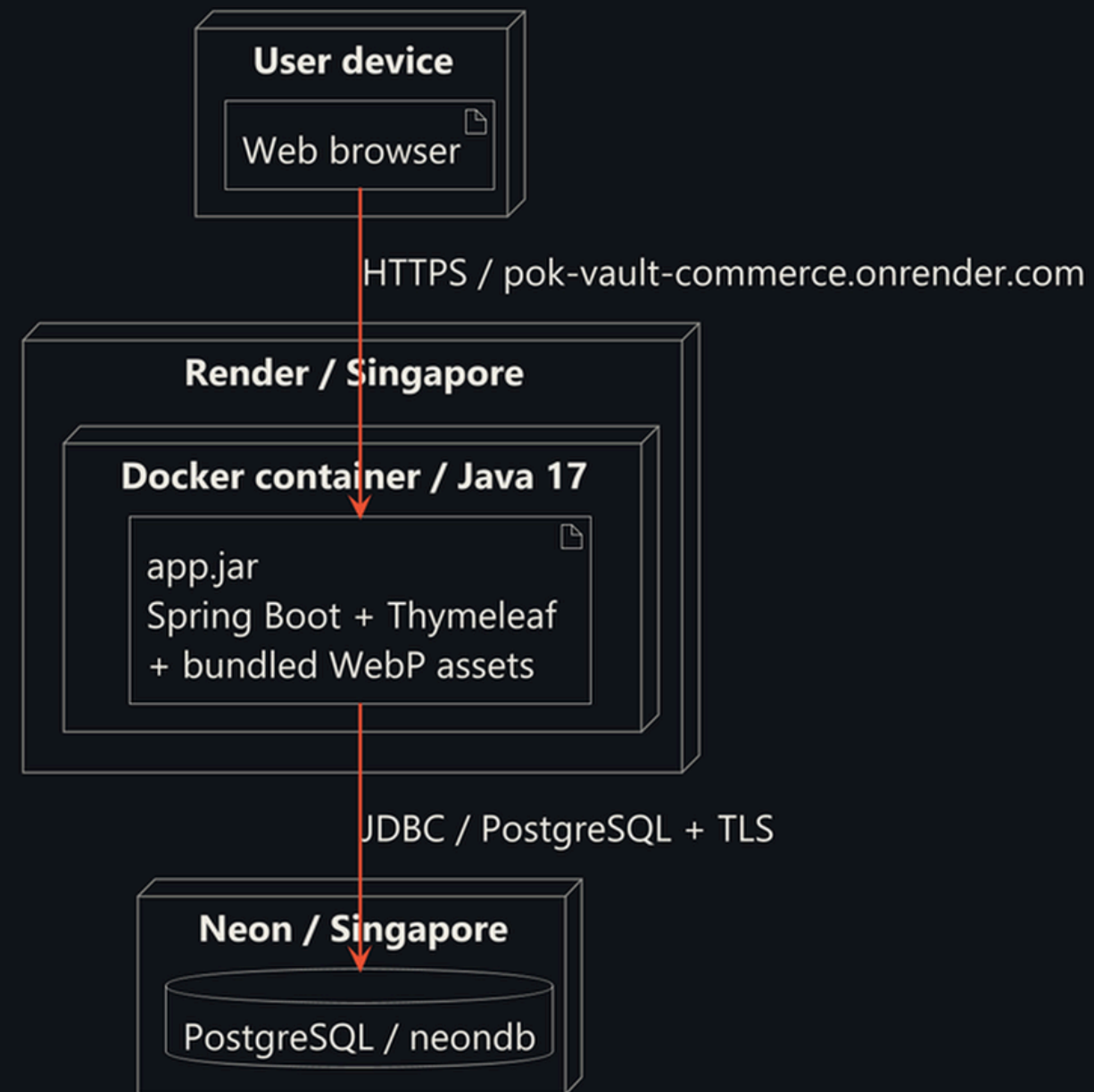
02 Observer รับเหตุการณ์

รับ Event ด้วย @EventListener
แล้วรวมสต็อกการ์ดแต่ละใบ

03 แจ้งเตือนผ่าน Log

เมื่อสต็อกคงเหลือไม่เกิน 2
ระบบบันทึกค่าเตือนด้วย log.warn()

Component, Deploy และ Git



GitHub Actions: TEST / BUILD / GHCR • Render deploy แยกต่างหาก

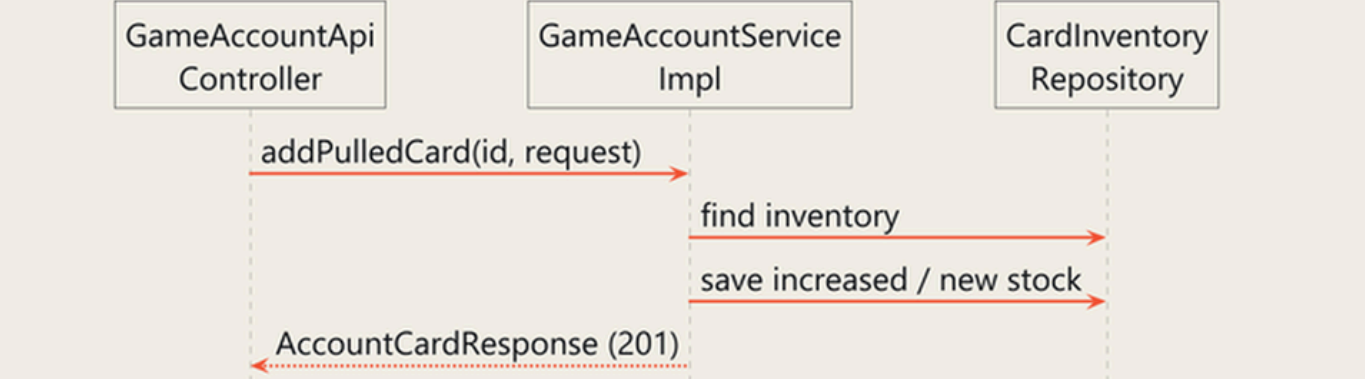
Repository: code / test / doc / img • Branch: name_id_section • Merge ผ่าน PR / reviewer

ภาพย่อ: ลำดับงาน 3 Scenarios

01 จองการ์ด



02 บันทึกเปิดซอง



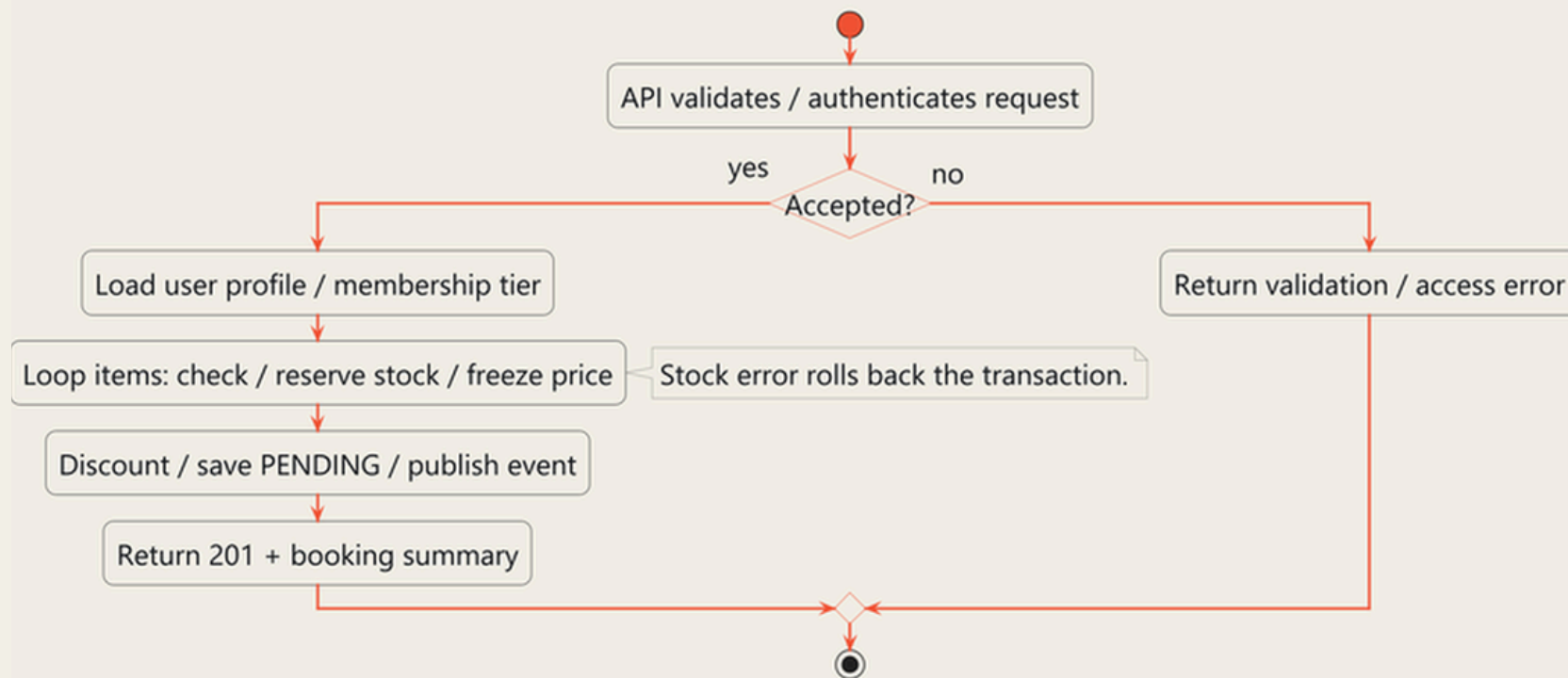
03 บันทึกเทรด



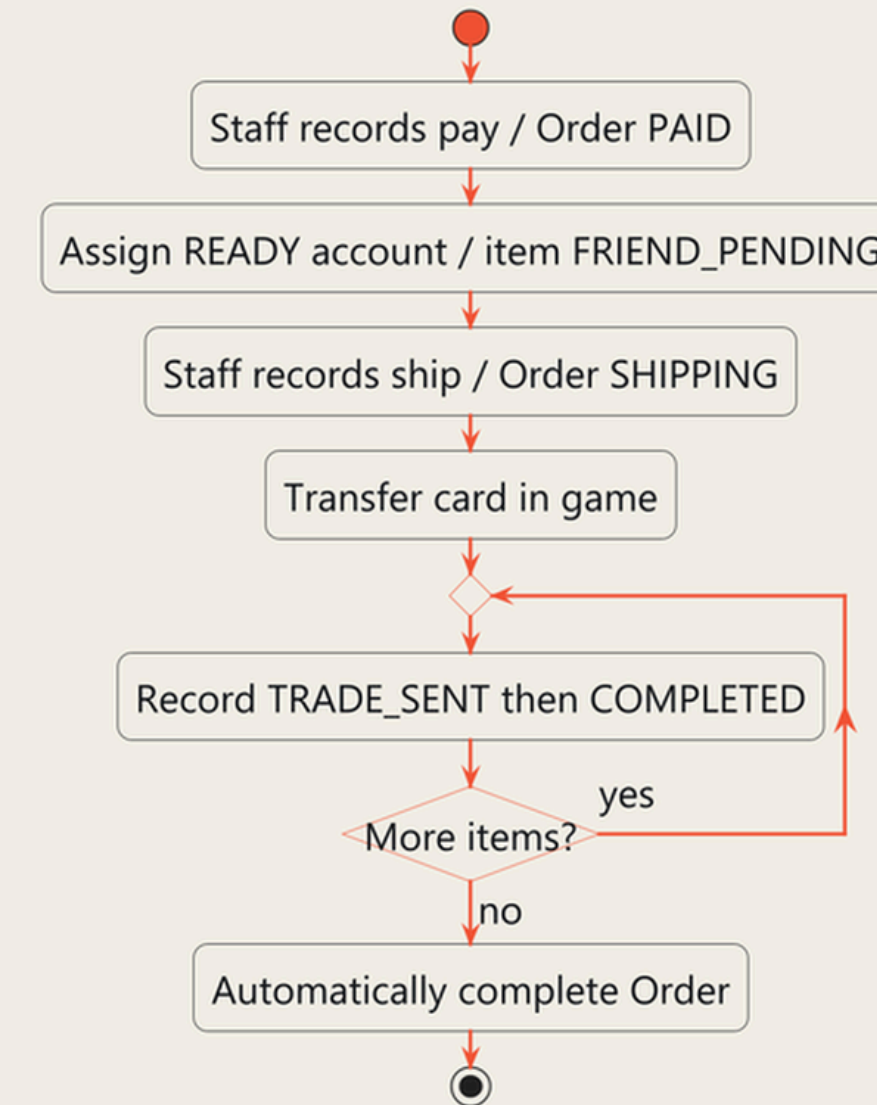
แต่ละภาพย่อเฉพาะขั้นตอนหลัก • รายละเอียด Sequence อยู่หน้า 34–36

Activity: จองและเทรด

01 BOOKING



02 FULFILLMENT



Action เป็นกล่องมุมโค้ง • Diamond = decision / merge • จุดทึบ = start และวงกลมคู่ = final

REST API and Swagger

RESOURCE	GET	POST	PUT	DELETE
/api/v1/cards	200	201	200	204
/api/v1/accounts	200	201	200	204

Validation 400 · NotFound 404 · Conflict 409 · Global Exception Handler

Pagination + Sorting

[/swagger-ui/index.html
/v3/api-docs](/swagger-ui/index.html/v3/api-docs)

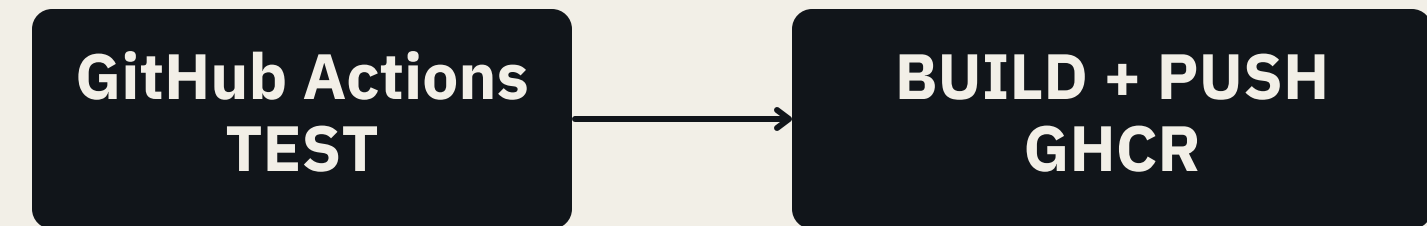
Testing และ Deployment

373

Tests ผ่านทั้งหมด

0 failures / 0 errors / 0 skipped

JUnit 5 + Mockito + Spring Boot Test
Local Java 21 / H2 PostgreSQL mode
ไม่ใช่ cloud end-to-end



Render deploy แยกต่างหาก
Neon PostgreSQL

TEST-REPORT.md • commit 82c449b

Testing: สิทธิผู้ใช้ในโค้ดจริง

ApiAuthorizationTest.java

L41-45

```

41 @Autowired MockMvc mvc;
42 @Autowired UserRepository users;
43 @Autowired PasswordEncoder encoder;
44 @MockitoBean OrderService orders;
45 @MockitoBean CardService cards;

```

ApiAuthorizationTest.java

L110-114

```

110 @Test void customerCannotSpoofAnotherCustomerId() throws Exception {
111     mvc.perform(post("/api/v1/orders").with(user(red.getUsername()).roles("CUSTOMER"
112                                     ")))
113         .contentType(MediaType.APPLICATION_JSON).content(booking(blue.getId())))
114         .andExpect(status().isForbidden());
115     verifyNoInteractions(orders);

```

01 Spring Boot Test

ใช้ Security Filter Chain จริง
Mock Service ด้วย @MockitoBean

02 จำลองผู้ใช้ CUSTOMER

ส่งคำสั่งซื้อโดยใช้ userId ของคนอื่น
ผ่าน MockMvc

03 ตรวจสอบผลและการเรียก Service

ต้องได้ HTTP 403
และ verifyNoInteractions(orders)

ตัวอย่างส่วนของสิทธิบัตร: ระบบปฏิเสธการส่งจองแทนลูกค้าคนอื่นก่อนเรียก Service

สิ่งที่ เรียนรู้

01 UI ต้องอัปเดตตาม state เมื่อ item สุดท้ายทำให้ออเดอร์ปิดอัตโนมัติ

02 repository code / test / doc / img ต้องจัดและแก้ path ให้ตรง

110.96 MB → 10.06 MB

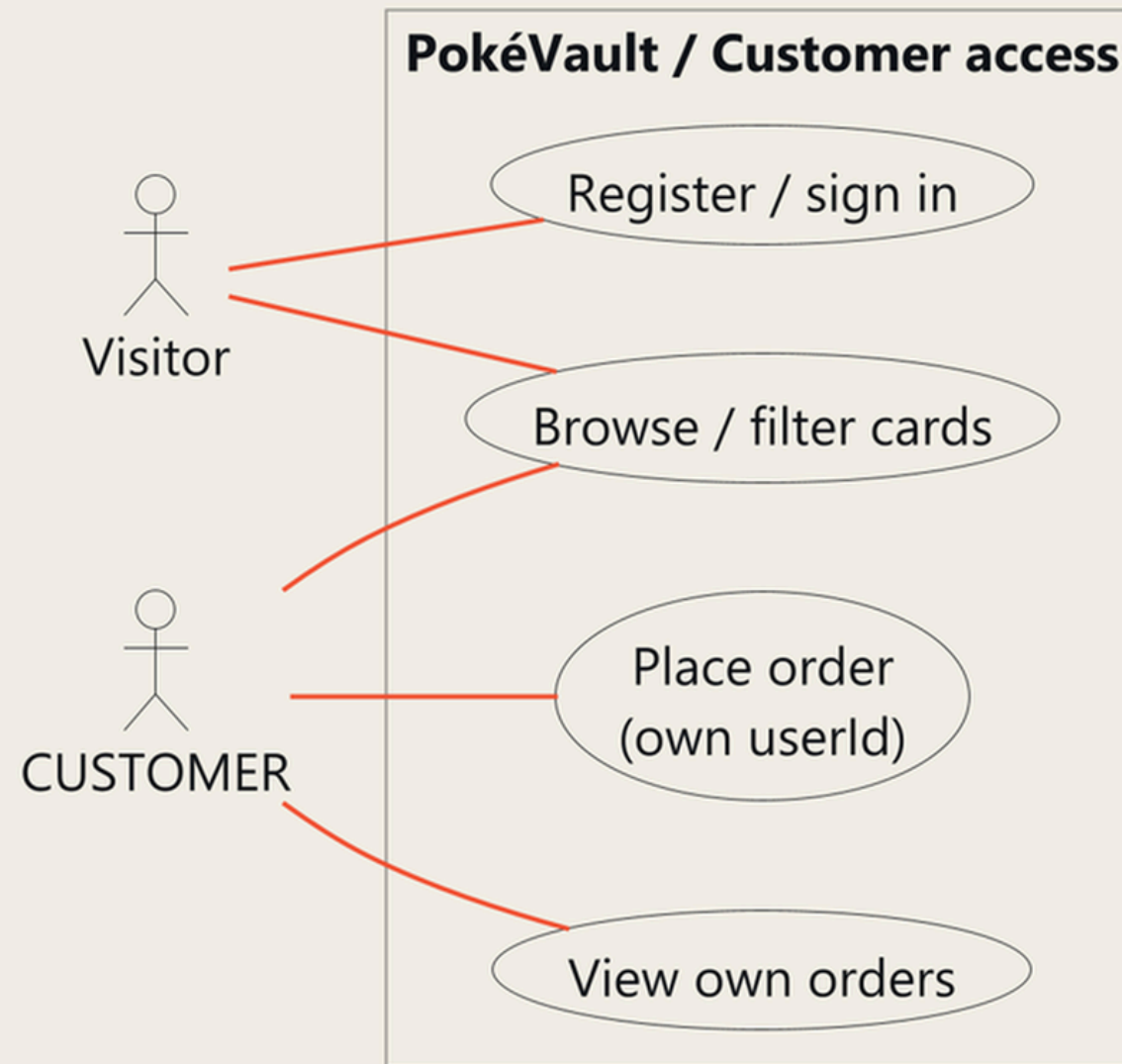
WebP เล็กลง 90.93% • 86 A1a assets • seed catalog ที่เชื่อมโยงจริง 4 รายการ

ศึกษา core/catalog • สัพพัญญู inventory • ธนภูมิ orders/discount
แทนคุณ state/trade • สรวิชญ์ frontend/devops

pok-vault-commerce.onrender.com

Use Case และสิทธิ์ผู้ใช้

CUSTOMER / PUBLIC

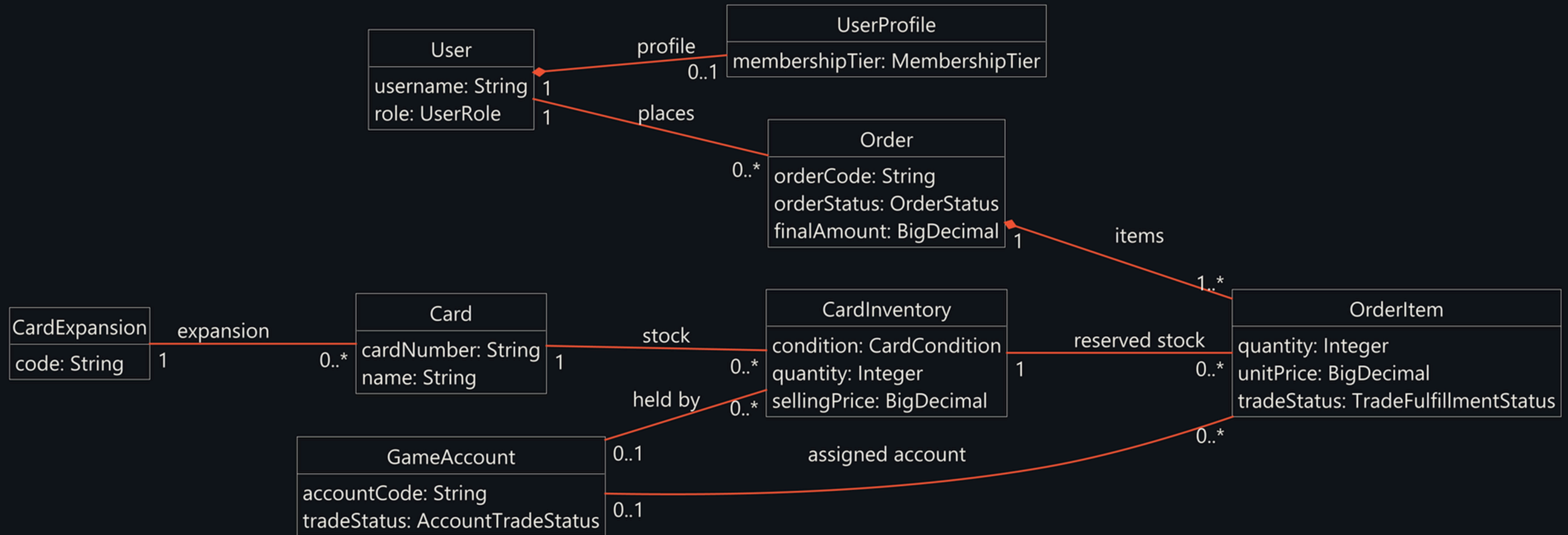


STORE OPERATIONS



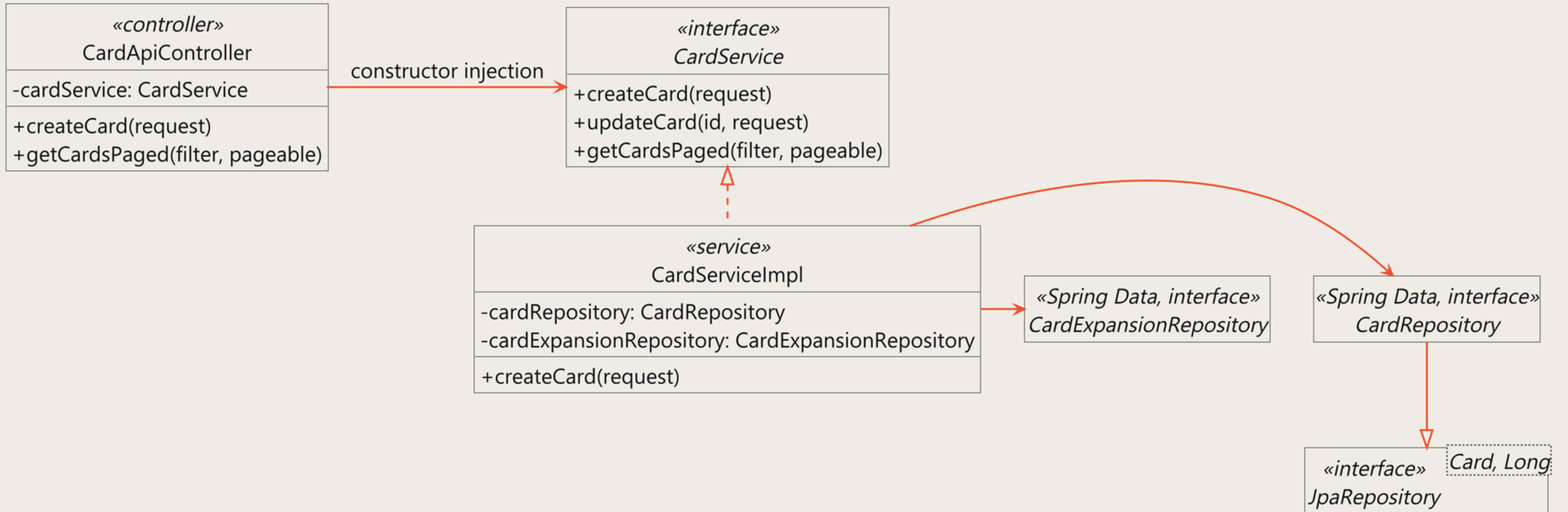
STAFF และ ADMIN ใช้สิทธิ์ชุดเดียวกันตาม SecurityConfig • ระบบรับเงิน/ส่งการ์ดจริงอยู่นอกเว็บ

Domain Model: สิ่งในระบบจัดการ



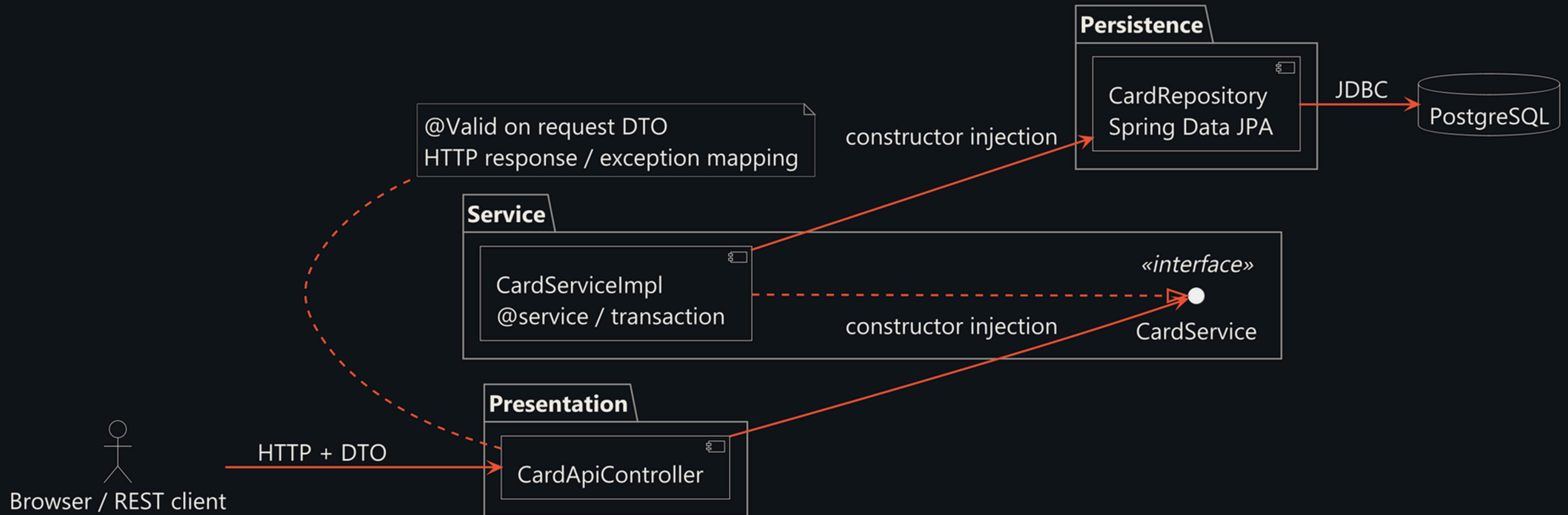
ภาพย่อ entity หลัก • Order items ≥ 1 เป็นกฎ API • ภาพ ER แสดงชั้นต่ำตาม SQL เป็น 0..*

Class Diagram: Catalog



เส้นประ + สามเหลี่ยมโปรง = implements • เส้นทึบ + สามเหลี่ยมโปรง = extends

Layered Architecture: Request จริง



POST /api/v1/cards • Request DTO / @Valid / Service / Repository / Response DTO (201)

SOLID: S, O และ L

S

แยกหน้าที่ของคลาส

WebViewController → HTTP /
Model
WebPageServiceImpl → View data
LowStockObserver → ตรวจสอบสต็อก

O

เพิ่มสูตรส่วนลดผ่าน Strategy

DiscountService รับ List<Strategy>
เลือกด้วย supports(tier)
เพิ่ม implementation ตามกติกา

L

เรียกใช้ผ่านสัญญาเดียวกัน

Regular / Vip / Wholesale ใช้
calculate(BigDecimal)
supports · getDiscountPercentage

ส่วนลดจากยอด 1,500: REGULAR 0 · VIP 150 · WHOLESALE 225 บาท

SOLID: I และ D

I

แยก Interface ตามงาน

CardService → แกดतालोक

GameAccountService → บัญชี / เปิดซอง

WebPageService → ข้อมูลสำหรับหน้าเว็บ

D

พึ่ง Interface ผ่าน Constructor

CardApiController

↓ CardService «interface»

↑ CardServiceImpl

↓ Repository «interface»

@RequiredArgsConstructor

private final CardService cardService;

Mock dependency ได้ตรงจุด • ตัวอย่างนี้ไม่อ้างว่าทุกคลาสในระบบทำ SOLID สมบูรณ์

SOLID: Interface และ Constructor

CardApiController.java L23-25, 27-29

```

23 @RestController
24 @RequestMapping("/api/v1/cards")
25 @RequiredArgsConstructor
   บรรทัด 26-26 ไม่แสดงในภาพนี้
27 public class CardApiController {
28
29 private final CardService cardService;
```

CardService.java L12-14, 28-33

```

12 public interface CardService {
13
14 List<CardResponse> getAllCards(CardFilterRequest filter);
   บรรทัด 15-27 ไม่แสดงในภาพนี้
28 CardResponse createCard(CardRequest request);
29
30 CardResponse updateCard(Long id, CardRequest request);
31
32 void deleteCard(Long id);
33 }
```

01 D: พึงพา Interface

Controller รับ CardService
ซึ่งเป็น Interface ของโมดูล Catalog

02 Constructor Injection

@RequiredArgsConstructor สร้าง
Constructor สำหรับ final field

03 I: แยกสัญญาณตามโมดูล

CardService รวมงานของ Catalog
แยกจาก Vault และ Trade Service

เปลี่ยน implementation หรือใช้ Mock ได้ผ่านสัญญาณ CardService ที่ Controller ใช้งาน

Design Patterns

STRATEGY

Regular 0% / VIP 10%
Wholesale 15%
DiscountStrategy
คำนวณด้วย BigDecimal

STATE

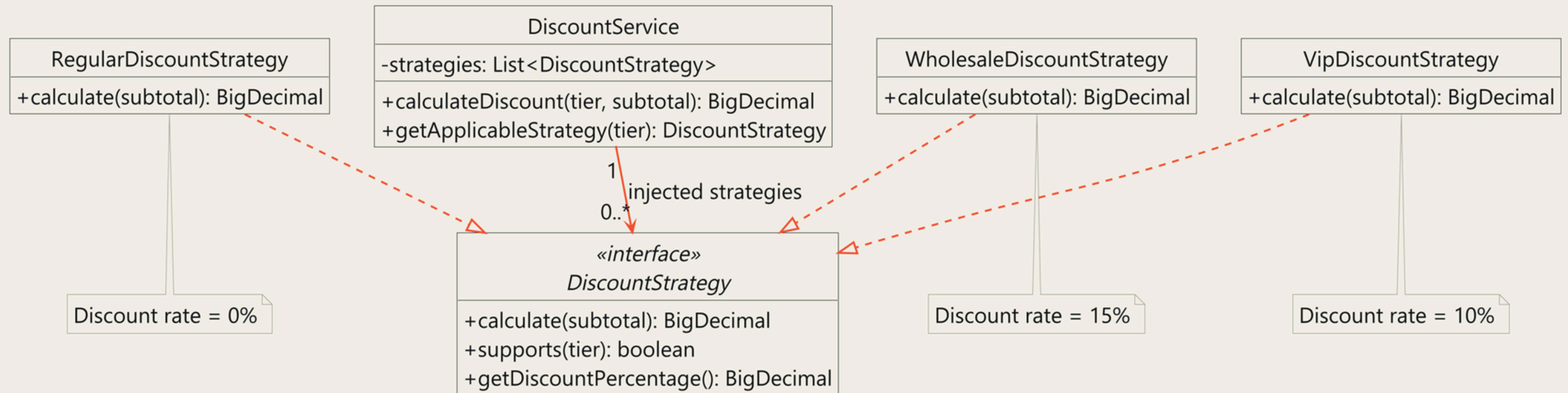
OrderContext + OrderState
บังคับ transition ที่อนุญาต

OBSERVER

OrderPlacedEvent
→ LowStockObserver
stock ≤ 2 → log เตือน
@EventListener

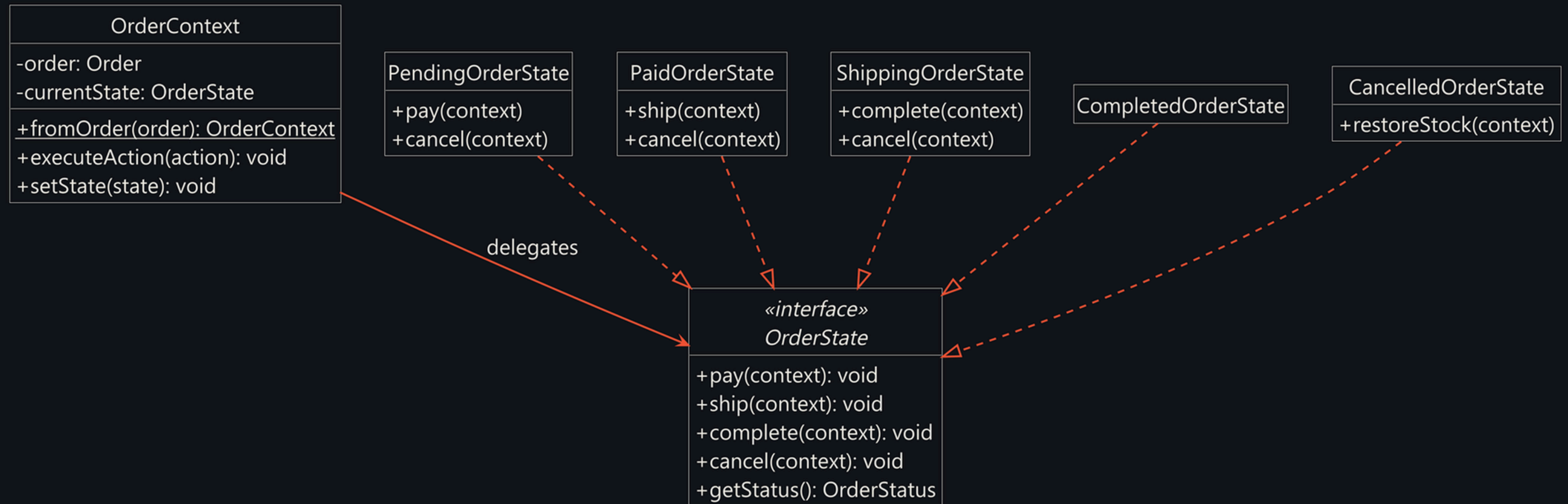
ทั้ง 3 แบบเป็น GoF Behavioral Patterns • [doc/design-patterns.md](#)

Strategy: สูตรส่วนลดสมาชิก



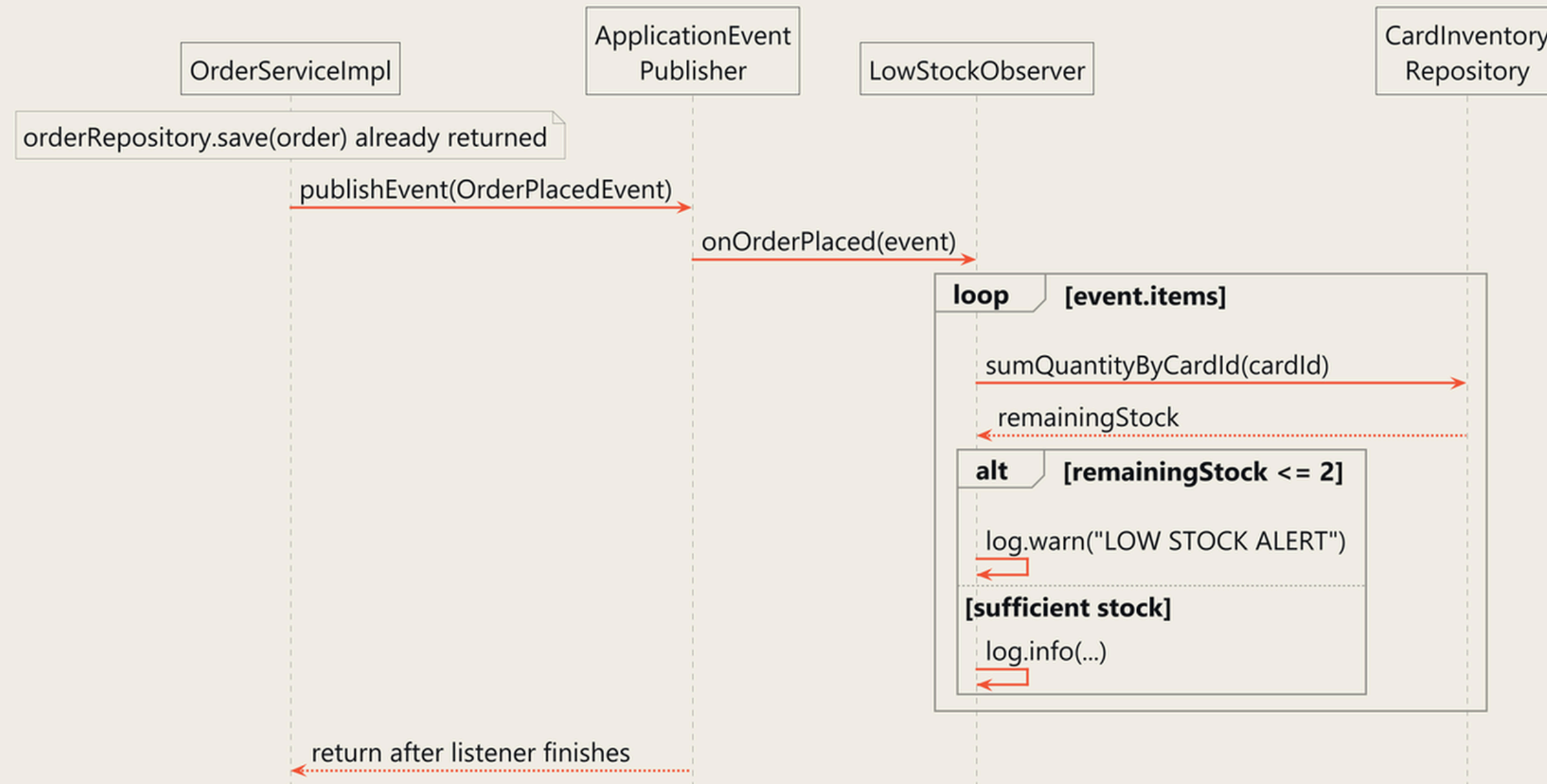
`supports(tier)` เลือก strategy จาก List ที่ inject • สูตรส่วนลด 0% / 10% / 15%

State: กฎอยู่ใน State ที่รับผิดชอบ



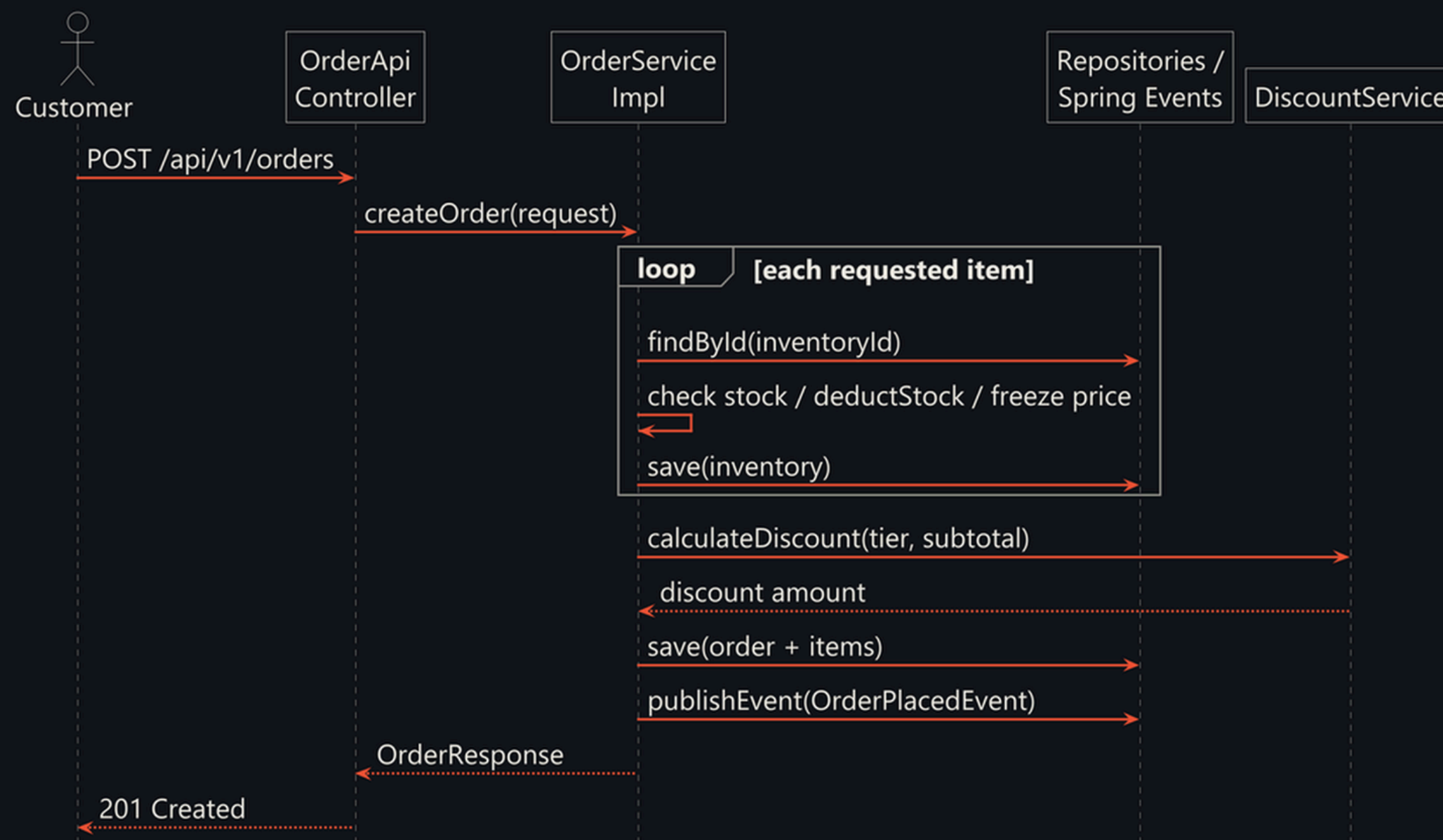
fromOrder ใช้ switch เลือก state • default action ที่ไม่อนุญาตโยน `InvalidOrderStateException`

Observer: ตรวจสอบต่อภายหลังสั่งจอง



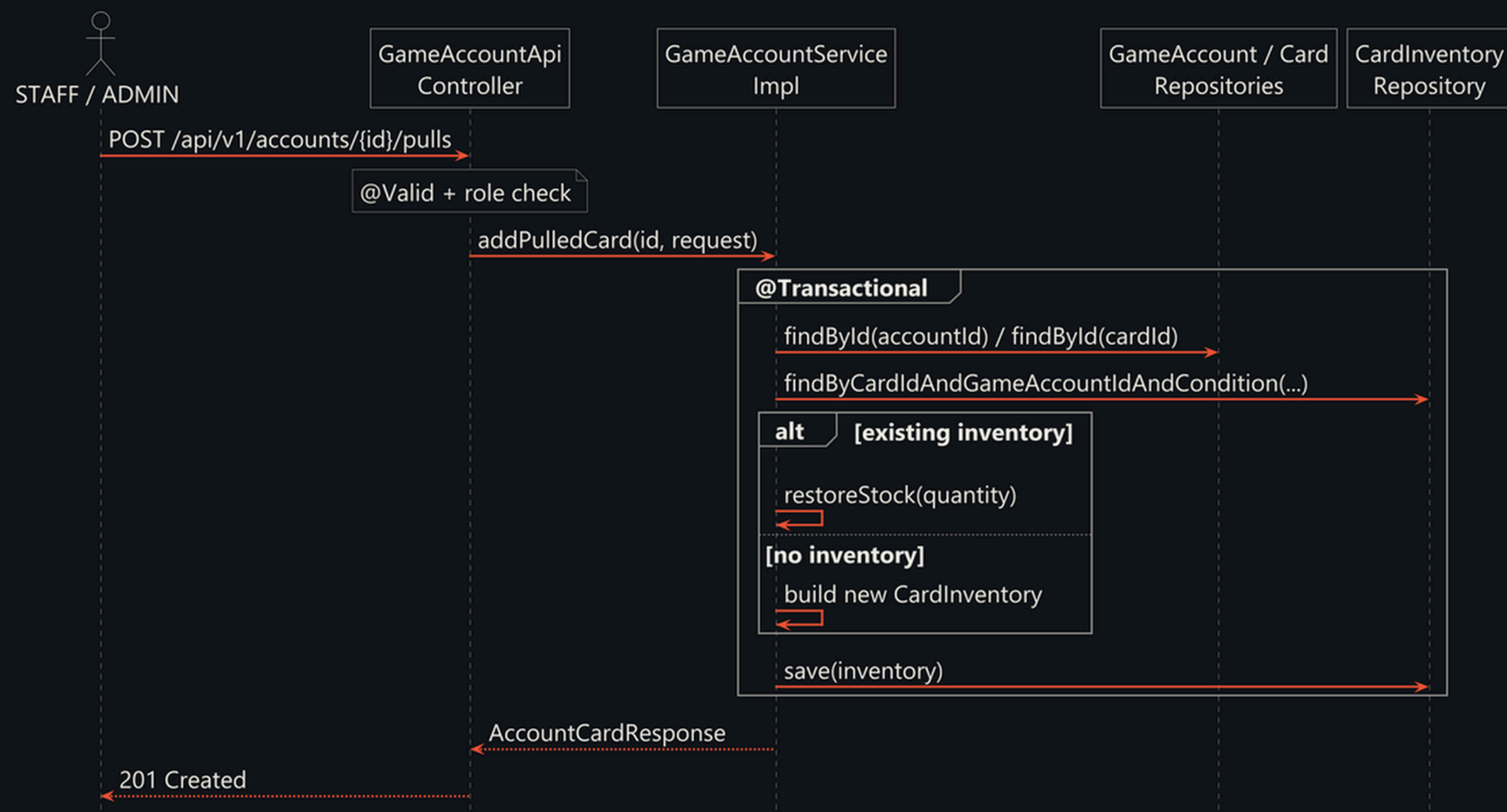
Synchronous listener ภายใน transaction ก่อน commit • remainingStock <= 2 เขียน WARN log

Sequence 01: สั่งจองการ์ด



@Valid / ตรวจสอบเจ้าของบัญชี / อ่าน tier จาก profile • @Transactional ครอบคลุมงานของ service

Sequence 02: บันทึกเปิดช่อง



Stock อ้างอิง card + account + condition • เพิ่มจำนวนเดิม หรือสร้าง inventory ใหม่

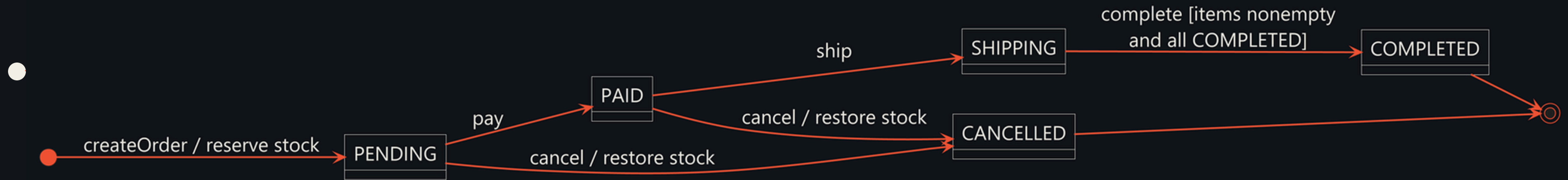
Sequence 03: จับคู่และส่งมอบ



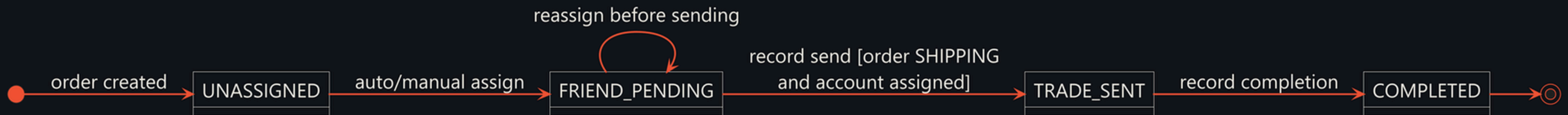
ย่อชั้น Controller / Repository • staff ส่งการ์ดในเกมก่อนบันทึก TRADE_SENT และ COMPLETED

State: ออเดอร์และรายการเทรด

ORDER

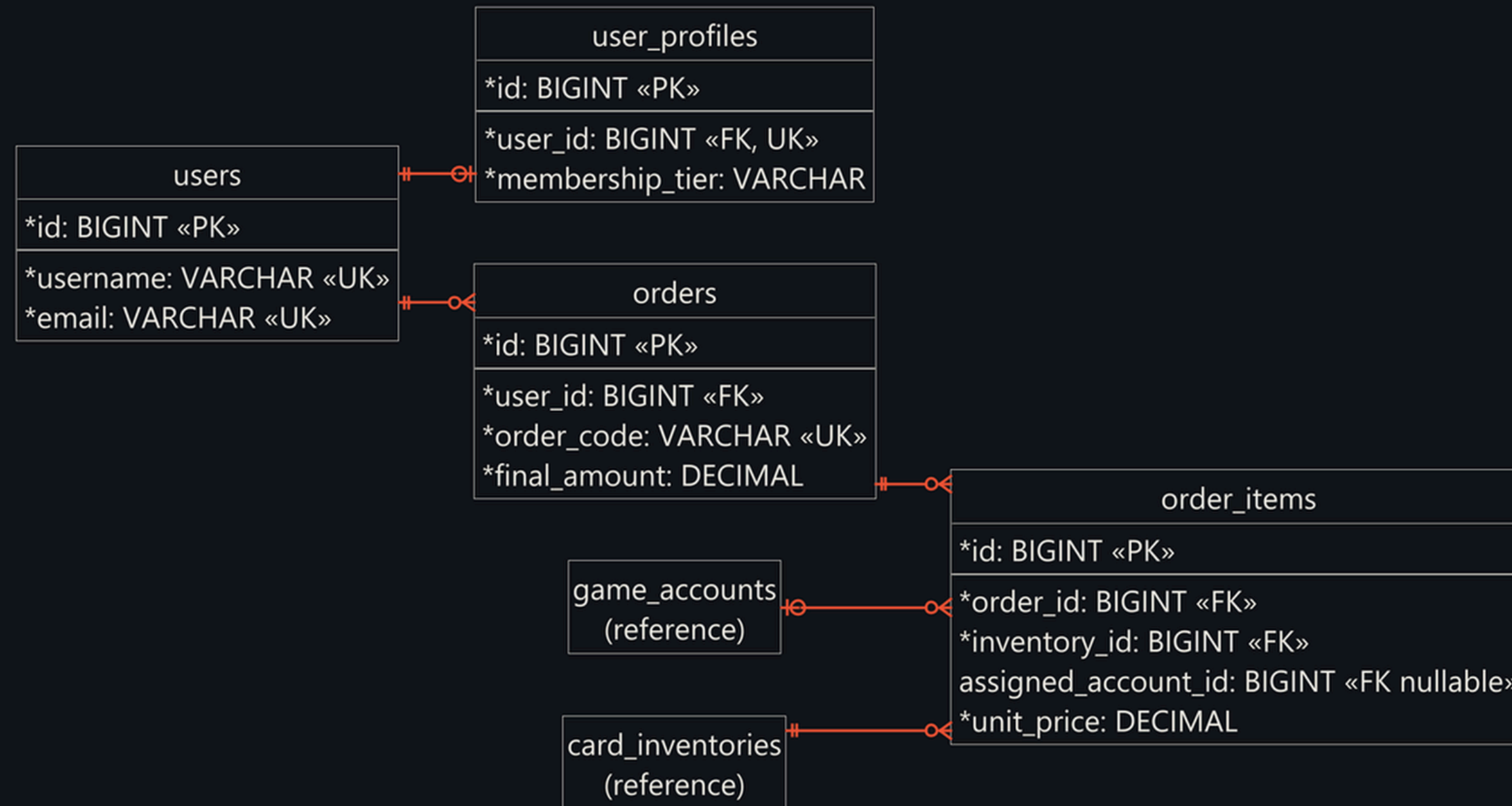


ORDER ITEM



ส่ง trade status เดิมซ้ำได้ • ห้ามข้าม FRIEND_PENDING / TRADE_SENT / COMPLETED

ER: Users และ Orders



Profile: UNIQUE user_id • OrderItem: snapshot unit_price • assigned_account_id nullable

Database: กฎข้อมูลและการเปลี่ยน Schema

CONSTRAINTS	CASCADE / FETCH	INDEX / MIGRATION
FK / NOT NULL / UNIQUE Money: DECIMAL(10,2) Inventory qty ≥ 0 · Item qty ≥ 1 unit_price / subtotal เก็บเป็น snapshot	User.profile / Order.items: Cascade.ALL + orphanRemoval ส่วนใหญ่ LAZY; Card.expansion EAGER SQL: CASCADE profile / items RESTRICT ข้อมูลอ้างอิง SET NULL บัญชี optional	PK / UNIQUE มี index รองรับ ยังไม่พบ explicit secondary index ใน code ปัจจุบัน schema.sql + data.sql core-schema-alignment.sql รันด้วยมือ ไม่ใช่ Flyway

แยกพฤติกรรม JPA Cascade ออกจาก SQL ON DELETE ให้ชัดเจน

Database: ความสัมพันธ์ใน JPA

Order.java

L57-59

```
57 @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true,
    fetch = FetchType.LAZY)
58 @Builder.Default
59 private List<OrderItem> items = new ArrayList<>();
```

OrderItem.java

L22-32

```
22 @ManyToOne(fetch = FetchType.LAZY)
23 @JoinColumn(name = "order_id", nullable = false)
24 private Order order;
25
26 @ManyToOne(fetch = FetchType.LAZY)
27 @JoinColumn(name = "inventory_id", nullable = false)
28 private CardInventory inventory;
29
30 @ManyToOne(fetch = FetchType.LAZY)
31 @JoinColumn(name = "assigned_account_id")
32 private GameAccount assignedAccount;
```

01 Order มีหลาย OrderItem

mappedBy = "order"

Cascade.ALL และ orphanRemoval

02 FK ตาม ER Diagram

order_id และ inventory_id

กำหนด nullable = false

03 บัญชีที่ใช้เทรดเป็นทางเลือก

assigned_account_id ยังว่างได้

และความสัมพันธ์ใช้ LAZY

JPA Cascade ควบคุมการทำงานผ่าน Entity ส่วน SQL ON DELETE เป็นกฎอีกระดับในฐานข้อมูล

REST CRUD: Card Catalog

METHOD	RESOURCE	STATUS	หน้าที่
GET	/api/v1/cards	200	รายการ + filter
GET	/api/v1/cards/{id}	200	รายละเอียด
POST	/api/v1/cards	201	สร้างการ์ด
PUT	/api/v1/cards/{id}	200	แก้ไขการ์ด
DELETE	/api/v1/cards/{id}	204	ลบ · body ว่าง

GET ดูการ์ดเป็น public • การเขียนใช้ STAFF / ADMIN

CardRequest: expansionId, cardNumber, name, cardType, rarity • รับผ่าน @Valid

REST CRUD: Game Accounts

METHOD	RESOURCE	STATUS	หน้าที่
GET	/api/v1/accounts	200	รายการบัญชี
GET	/api/v1/accounts/{id}	200	รายละเอียด
POST	/api/v1/accounts	201	สร้างบัญชี
PUT	/api/v1/accounts/{id}	200	แก้ไขบัญชี
DELETE	/api/v1/accounts/{id}	204	ลบบัญชีที่ไม่ถูกอ้างอิง

STAFF / ADMIN • POST /{id}/pulls → 201 • GET /{id}/cards → 200

PATCH /{id}/trade-status?status=READY → 200 • ลบไม่ได้เมื่อมี Inventory / Order อ้างอิง

Validation และ Error Contract

VALIDATE REQUEST

@NotNull expansionId / cardType / rarity
@NotBlank name / cardNumber
@Size name ≤ 100 · cardNumber ≤ 20
hp: @Min(0) · cost: @DecimalMin(0)
Friend ID: 16 digits / hyphen groups

ตัวอย่าง ErrorResponse

status: 400 · error: VALIDATION_FAILED
message · timestamp · path
details: { name: "must not be blank" }

STATUS	ERROR CODE
400	VALIDATION_FAILED / INSUFFICIENT_STOCK
404	NOT_FOUND
409	STATE_CONFLICT
401 / 403	AUTHENTICATION_REQUIRED / ACCESS_DENIED
500	INTERNAL_SERVER_ERROR

REST: Validation และ HTTP Status

CardApiController.java

L83-89

```
83 @PostMapping
84 @Operation(summary = "Create a new card", description = "Add a new Pokemon card
to the catalog")
85 public ResponseEntity<ApiResponse<CardResponse>> createCard(@Valid @RequestBody
CardRequest request) {
86     CardResponse created = cardService.createCard(request);
87     return ResponseEntity.status(HttpStatus.CREATED)
88         .body(ApiResponse.ok("Card created successfully", created));
89 }
```

GlobalExceptionHandler.java

L79-86

```
79 ErrorResponse errorResponse = ErrorResponse.builder()
80     .timestamp(LocalDateTime.now())
81     .status(HttpStatus.BAD_REQUEST.value())
82     .error("VALIDATION_FAILED")
83     .message("Validation error in request payload")
84     .path(request.getRequestURI())
85     .details(fieldErrors)
86     .build();
```

01 ประกาศ Resource และเอกสาร

@PostMapping รับคำสั่งสร้างการ์ด
@Operation อธิบายใน Swagger

02 รับข้อมูลด้วย @Valid

ตรวจสอบ CardRequest ก่อนเข้า Service
สร้างสำเร็จตอบ HTTP 201

03 Validation Error ตอบ 400

ErrorResponse มี
VALIDATION_FAILED
และ details ระบุ field ที่ผิด

ผลสำเร็จและข้อผิดพลาดมีโครงสร้างชัดเจน ผู้ใช้ API ตรวจสอบทั้ง HTTP Status และ Response Body

Pagination, Sorting และ Filtering

GET `/api/v1/cards/paged`

`?page=0&size=10&sort=cardNumber,asc&expansionCode=A1a`

REQUEST

Default: page 0 · size 10

sort cardNumber ASC

Filters: name / expansionCode / rarity

cardType / elementType

Search: `/api/v1/cards/search?q=Mew`

RESPONSE · `ApiResponse.data`

`content: [...]`

`pageNumber · pageSize`

`totalElements · totalPages`

`first · last`

JSON ใช้ first / last ตามผลตอบแทนจริง

ชื่อ field ใน Java คือ isFirst / isLast

Swagger: อ่านและทดลอง Contract

01 เปิด Swagger UI บน Deploy URL

`pok-vault-commerce.onrender.com/swagger-ui/index.html`

02 Card Catalog API → GET `/api/v1/cards/paged`

Try it out: page 0 · size 10 · sort cardNumber,asc

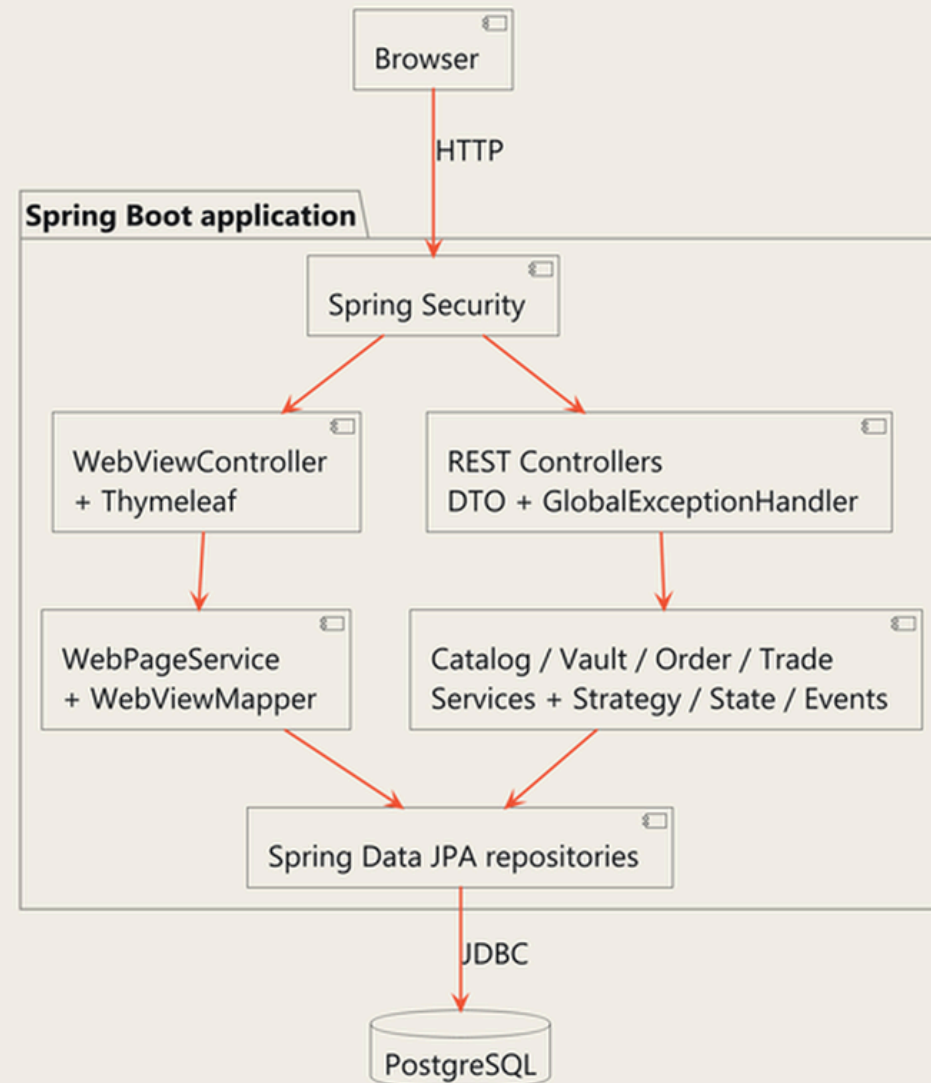
03 Execute → ตรวจ 200 + Response Schema

OpenAPI JSON: `/v3/api-docs` • @Tag / @Operation ระบุ resource

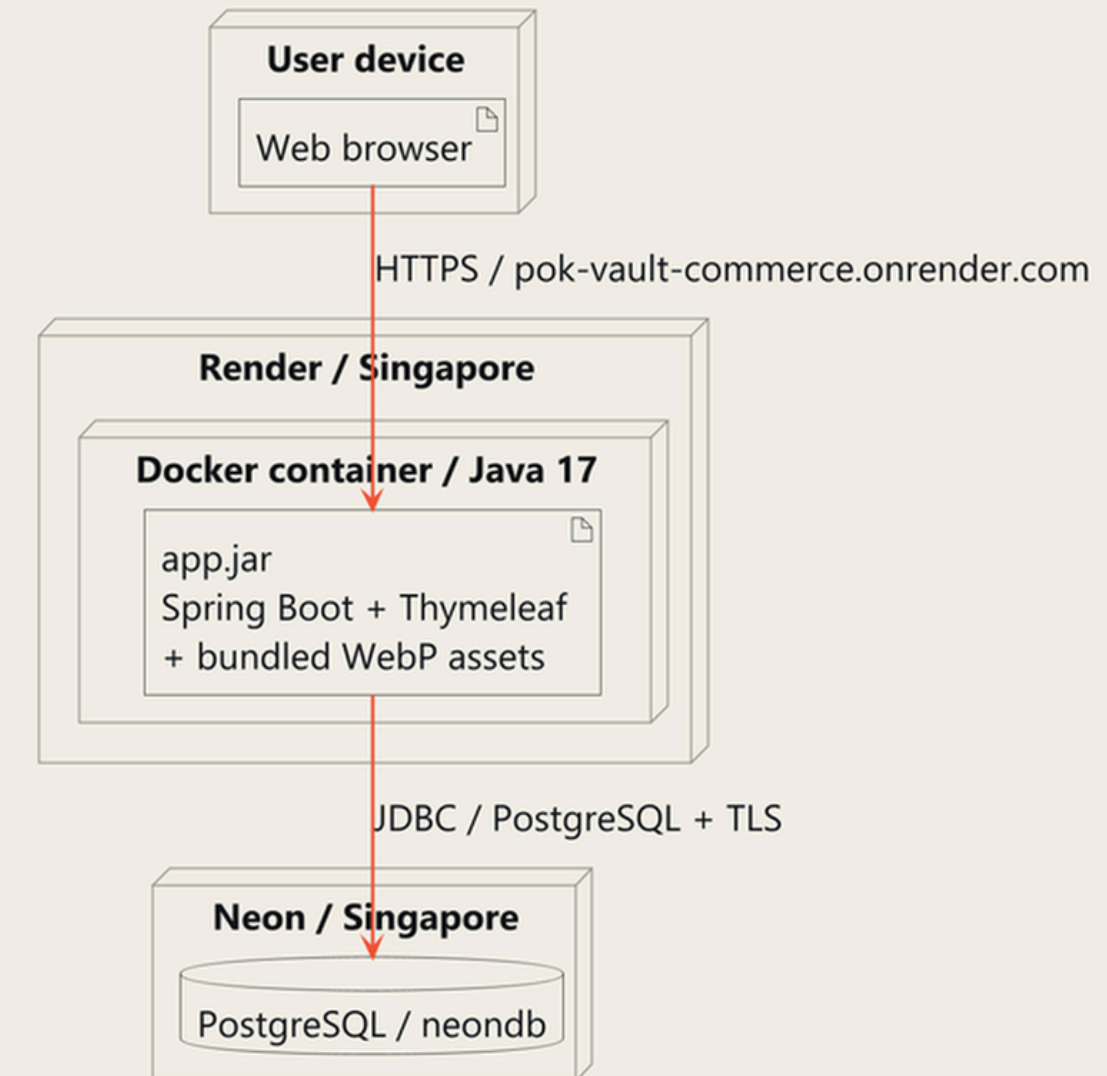
Docs เปิดดูได้ • API ที่ป้องกันต้อง Login ด้วย session • ระบบนี้ไม่ได้ใช้ Bearer JWT

Component และ Deployment

COMPONENT



DEPLOYMENT



Runtime แยกจาก Build/CI • Render รัน app.jar • ข้อมูลการอยู่ใน Neon PostgreSQL

LIVE DEMO

pok-vault-commerce.onrender.com

จองการ์ด 80s — ธนภูมิ
เพิ่มสต็อก 45s — สัพพัญญู
เทรดครบ 60s — แทนคุณ
Swagger 25s — สรวิชญ์

Pokémon TCG Pocket Gallery

หน้าแรก

แกลเลอรีการ์ด

คลังสินค้า

บัญชีเกม

คำสั่งซื้อ

สวัสดี, staff_ash

ออกจากระบบ

คำสั่งซื้อและการเทรดในเกม

จัดการสถานะคำสั่งซื้อ (Pending → Paid → Shipping → Completed) และจับคู่บัญชีเกมเพื่อเทรดการ์ดให้ลูกค้าผ่าน Friend Code

+ สร้างคำสั่งซื้อที่คลังสินค้า

รหัสคำสั่งซื้อ	การ์ด	ลูกค้า / FRIEND ID	สถานะคำสั่งซื้อ	การเทรดในเกม	ยอดชำระ	จัดการสถานะ
ORD-2026-004 2026-10-09 00:34	 Venusaur ex A1 #004/226 x1	Red Champion VIP 1234-5678-9012-3456 IGN: RedMaster	PENDING → PAID → SHIPPING → COMPLETED	จัดการเทรด รายการใบออเดอร์	\$162.00 ประหยัด \$18.00	เทรดแล้ว
ORD-2026-003 2026-10-09 00:34	 Zapdos ex A1 #104/226 x1	Red Champion VIP 1234-5678-9012-3456 IGN: RedMaster	PENDING → PAID → SHIPPING → COMPLETED	จัดการเทรด รายการใบออเดอร์	\$162.00 ประหยัด \$18.00	เทรดสำเร็จ
ORD-2026-002 2026-10-09 00:34	 Mewtwo ex A1 #129/226 x1	Red Champion VIP 1234-5678-9012-3456 IGN: RedMaster	PENDING → PAID → SHIPPING → COMPLETED	จัดการเทรด รายการใบออเดอร์	\$225.00 ประหยัด \$25.00	เริ่มเทรด ยกเลิก
ORD-2026-001 2026-10-09 00:34	 Pikachu ex A1 #096/226 x1	Red Champion VIP 1234-5678-9012-3456 IGN: RedMaster	PENDING → PAID → SHIPPING → COMPLETED	จัดการเทรด รายการใบออเดอร์	\$198.00 ประหยัด \$22.00	ชำระเรียบร้อยแล้ว ยกเลิก

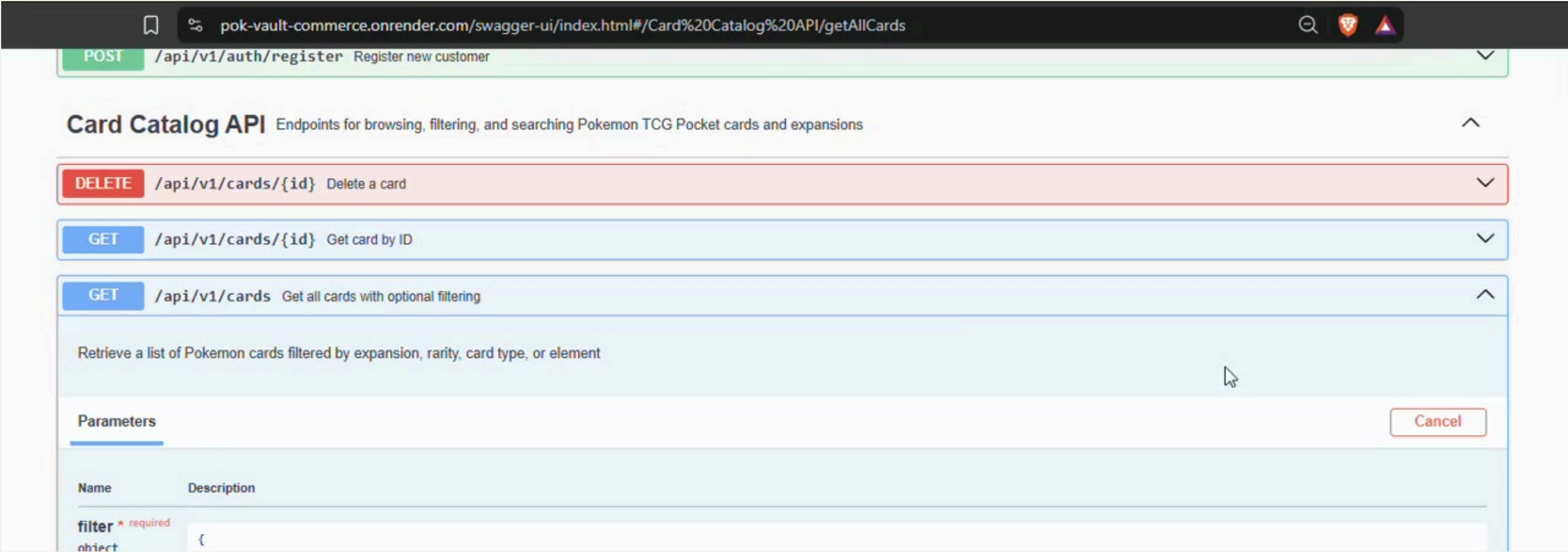
REST API and Swagger

RESOURCE

/api/v1/cards

GET

200



REST API and Swagger

RESOURCE

/api/v1/accounts

GET

200

